

Threads

François Trahay



Threads

Contexte d'exécution d'un processus

- Contexte: contexte d'exécution + contexte du noyau
- Espace mémoire: code, données, pile

Process context:

Execution context

Data registers {a0=147, a1=0x66, ...}

Stack pointer {sp=0x7ffffffd678}

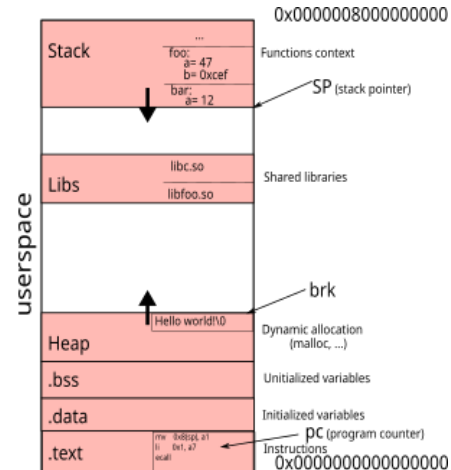
Program counter {pc=0x7ffff7e8f0d0}

Kernel context

Virt. Mem. structures

Descriptor table

brk pointer



Cloner un processus

- `fork()` crée un nouveau process et duplique le processus appelant
 - Contexte: contexte d'exécution + contexte du noyau
 - Sauf pour le registre `rax` (qui stocke la valeur de retour de `fork`)
 - Espace mémoire: code, données, pile

Process 1 context:

Execution context

Data registers (a0=147, a1=0x66, ...)

Stack pointer (sp=0x7fffffd678)

Program counter (rip=0x7ffff7e8f0d0)

Kernel context

Virt. Mem. structures

Descriptor table

brk pointer

Process 2 context:

Execution context

Data registers (a0=0, a1=0x66, ...)

Stack pointer (sp=0x7fffffd678)

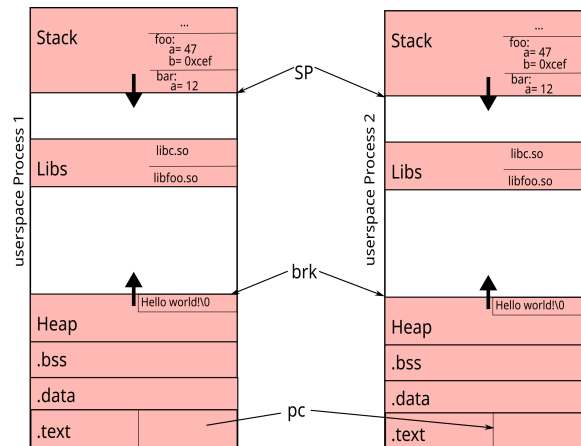
Program counter (rip=0x7ffff7e8f0d0)

Kernel context

Virt. Mem. structures

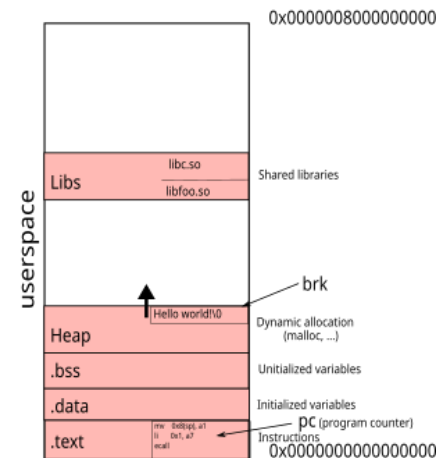
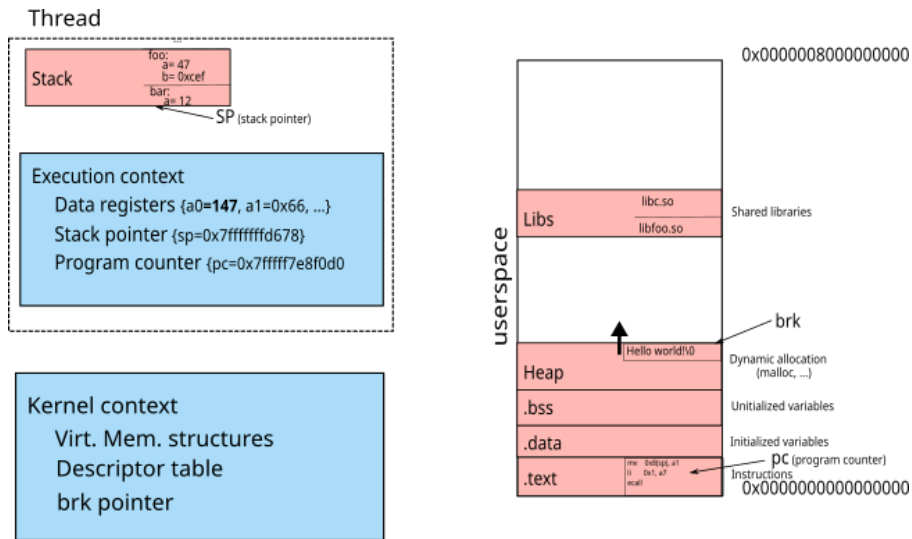
Descriptor table

brk pointer



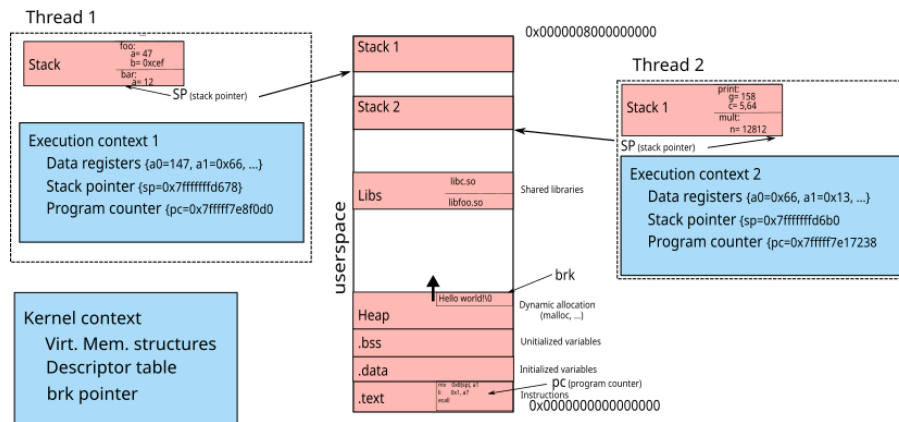
Flux d'exécution

- Une autre manière de représenter un processus est de séparer le flux d'exécution et les ressources du processus
 - Flux d'exécution (ou thread) : contexte d'exécution + pile
 - Ressources: code, données, contexte du noyau



Processus multi-thread

- Plusieurs flux d'exécution
- Des ressources partagées



Créer un pthread

```
int pthread_create(pthread_t *thread,  
                  const pthread_attr_t *attr,  
                  void *(*start_routine) (void *),  
                  void *arg);
```

- Crée un nouveau thread qui exécute la fonction `start_routine(arg)`
- `attr` (in): attributs du thread à créer
- `start_routine` (in): adresse de la fonction à exécuter par le nouveau thread
- `arg` (in): paramètre à passer à la fonction `start_routine`
- `thread` (out): identifiant du thread créé

Terminaison d'un thread

Un thread se termine quand

- il atteint la fin de sa fonction
- il appelle `void pthread_exit(void* retval)`

```
void* thread_function(void* arg) {  
    ...  
    if(...) {  
        ...  
        pthread_exit(NULL); // fin du thread  
    }  
    ...  
    return NULL; // fin de thread_function, donc destruction  
du thread  
}
```

Pthread_join

```
int pthread_join(pthread_t tid, void **retval);
```

- Attend la terminaison du thread `tid`.
- La valeur de retour du thread est stockée dans `*retval`

Partage de données

- L'espace mémoire est partagé entre les threads, notamment:
 - les variables globales
 - les variables static local
 - le contexte du noyau (descripteurs de fichiers, flux, signaux, etc.)
- Quelques ressources ne sont pas partagées
 - les variables locales
 - les registres

Accès concurrent aux données

- Plusieurs threads accèdent au même espace mémoire de manière concurrente
- Les structures de données partagées doivent être partagées
 - utilisation de sémaphores ou autres mécanismes (mutex, etc.)

Mutex

Assure l'accès en exclusion mutuelle d'une section critique.

```
pthread_mutex_t lock;  
pthread_mutex_init(&lock, NULL); // Initialise le verrou  
  
pthread_mutex_lock(&lock); // Prend le verrou  
counter++;  
pthread_mutex_unlock(&lock); // Relache le verrou  
  
pthread_mutex_destroy(&lock); // Détruit le verrou
```